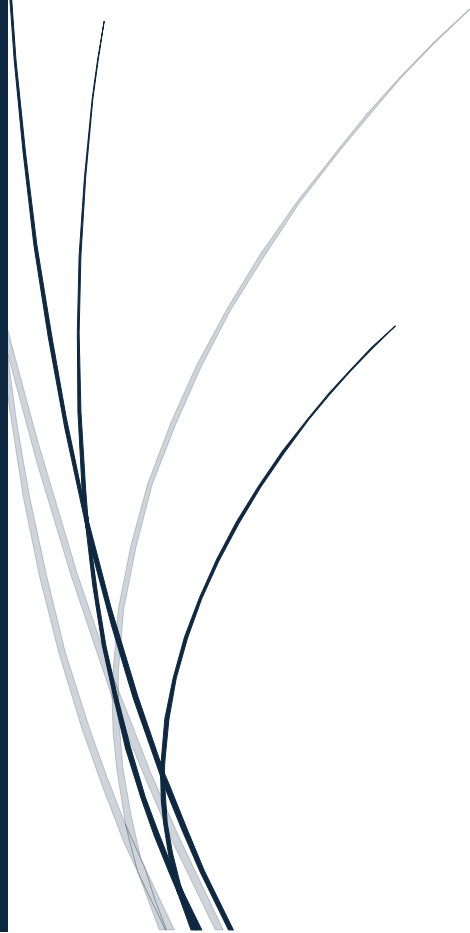


A thick dark blue vertical bar runs down the left side of the page. A dark blue arrow points to the right from this bar, containing the date.

01/06/2025

Rapport de stage

1^{re} Année EPSI La Défense

Several thin, curved lines in dark blue and light grey originate from the bottom left and sweep upwards and to the right.

Yacine Touazi

OMEGA BUSINESS INTERNATIONAL (OBI)

Sommaire

1. Glossaire des termes clés.....	3
2. Présentation de l'entreprise.....	5
2.1. Historique et spécialisation.....	5
2.2. Structure organisationnelle.....	6
2.3. Import et export des matériaux.....	7
2.4. Services complémentaires.....	8
2.5. Implantation géographique.....	9
3. Contexte du stage et objectifs.....	10
3.1. Contexte pédagogique et positionnement.....	10
3.2. Objectifs SMART du stage.....	11
4. Environnement technique et méthodologie.....	12
4.1. Plateformes hardware et architecture électronique.....	12
4.2. Logiciels, outils de développement et CI/CD.....	13
4.3. Méthodologie Agile et gestion de projet.....	14
5. Description détaillée des missions.....	15
6. Développement du contrôleur : approche générale et objectifs.....	16
6.1. HardwareController : abstraction des ressources matérielles.....	17
6.2. StateMachine : machine à états et conception orientée objet.....	18
6.3. PedestrianController : gestion asynchrone des demandes piétons...	19
6.4. Timer : temporisation et modes de veille.....	20
6.5. Code source complet et annotations.....	21
7. Déroulement des missions.....	22
7.1. Phase de spécification (Semaine 1).....	22
7.2. Phase de développement (Semaines 2 à 4).....	23
7.3. Phase de tests et validation (Semaines 5 et 6).....	24
8. Conclusion générale.....	25
9. Annexes complémentaires.....	26
10. Bibliographie et sources complémentaires.....	26
11. Remerciements.....	27

1. Glossaire des termes clés

Pour faciliter la lecture et la compréhension du rapport, voici les définitions des principaux termes et acronymes utilisés :

- **Arduino UNO** : Carte microcontrôleur open-source, idéale pour prototypage rapide. Modèle ATmega328P (16 MHz, 2 kB RAM, 32 kB Flash).
- **API (Application Programming Interface)** : Ensemble de fonctions et spécifications techniques permettant à deux logiciels de communiquer.
- **CI/CD (Continuous Integration / Continuous Deployment)** : Processus d'automatisation des tests, de l'intégration et du déploiement d'une application.
- **Firmware** : Logiciel embarqué sur un composant matériel, indispensable au fonctionnement du microcontrôleur.
- **GPIO (General Purpose Input/Output)** : Broches configurables en entrée ou sortie sur une carte microcontrôleur.
- **Machine à états** : Modèle algorithmique consistant à définir des états et transitions pour piloter un système.
- **Mode piéton** : Mode de fonctionnement spécifique qui déclenche un cycle prolongé pour la traversée des piétons.
- **Multimètre** : Outil de mesure électrique (tension, courant, résistance).
- **Oscilloscope** : Instrument permettant de visualiser des signaux électriques en fonction du temps.
- **Pull-up / Pull-down** : Résistances internes configurées pour assurer un état par défaut (haut ou bas) sur une entrée.
- **RSE (Responsabilité Sociétale des Entreprises)** : Ensemble des engagements d'une entreprise envers l'environnement et la société.

Chaque terme sera contextualisé dans les sections suivantes.

2. Présentation de l'entreprise

Omega Business International (OBI) est une entreprise française, créée il y a plus de cinq ans, spécialisée dans la distribution de métaux et profilés sur-mesure pour des clients tels qu'artisans, TPE/PME et industries. L'activité principale repose sur la fourniture de matériaux ferreux (inox, aciers divers) et non-ferreux (aluminium, cuivre, laiton, bronze), découpés à la demande et traités selon les spécifications exactes des utilisateurs. Plutôt que de s'imposer sur des volumes, OBI se distingue par une approche flexible : aucun minimum de commande n'est exigé et chaque profilé peut être préparé en suivant précisément les dimensions et finitions souhaitées. Cette agilité a rapidement séduit une clientèle variée désireuse de solutions personnalisées et réactives.

2.1. Historique et spécialisation

Fondée en 2019 par deux associés, OBI a débuté comme petit négociant de métaux ferreux, visant à répondre aux besoins locaux d'un marché qui manquait de réactivité pour les petites quantités. Très vite, l'activité s'est élargie aux alliages non-ferreux, avec l'introduction d'un service de découpe numérique permettant de proposer des pièces prêtes à l'emploi. Pour assurer la qualité et la précision, l'entreprise a investi dans :

- Une scie à ruban CNC, garantissant des coupes nettes au dixième de millimètre.
- Un centre d'usinage numérique capable de réaliser perçages, taraudages et profilés complexes.
- Un poste de traitement des surfaces proposant anodisation, galvanisation et laquage.

Parallèlement, OBI a mis en place une gestion informatisée (ERP/GPAO) pour suivre en temps réel les stocks et automatiser les réapprovisionnements selon des seuils prédéfinis.

2.2. Structure organisationnelle

OBI fonctionne avec une équipe réduite, composée de deux salariés permanents :

- **Directeur Financier** : responsable de la gestion administrative et financière de l'entreprise, y compris le suivi des comptes, l'analyse des marges et la négociation des conditions de paiement auprès des fournisseurs et banques. Il supervise également la tarification des produits en fonction des coûts d'achat et de production, et prépare les budgets annuels.
- **Directeur Commercial (Vente et Logistique)** : en charge de l'ensemble des activités commerciales et de la relation client. Il prospecte de nouveaux marchés, finalise les devis, assure le suivi des commandes, coordonne la préparation en atelier et la logistique d'expédition. Il gère également le service après-vente, centralise les retours clients et adapte l'offre selon les besoins du marché.

Cette organisation légère implique que le Directeur Financier et le Directeur Commercial collaborent étroitement : le financement des approvisionnements se fait en fonction des prévisions de vente, et chaque demande client est validée conjointement pour garantir la rentabilité et la réactivité. Leur polyvalence permet à OBI de traiter rapidement les commandes, d'ajuster les stocks et d'assurer un service de qualité sans couche hiérarchique supplémentaire.

2.3. Import et export des matériaux

Import et export des matériaux

Une part non négligeable de l'activité d'OBI repose sur des échanges internationaux. En effet, pour proposer une large gamme d'alliages de qualité, l'entreprise importe régulièrement des métaux depuis des fournisseurs européens et asiatiques reconnus : aciers inoxydables allemands, aluminium suédois ou cuivre japonais. Chaque importation suit un processus rigoureux :

1. **Sélection des fournisseurs** : seuls les partenaires certifiés ISO et respectant les normes EN ou ASTM sont retenus.
2. **Commandes groupées** : pour optimiser les coûts de transport, OBI planifie des réceptions groupées, puis redistribue les quantités selon les besoins clients.
3. **Formalités douanières** : chaque lot importé fait l'objet d'une déclaration d'importation (DAU) et d'un contrôle documentaire garantissant la conformité aux normes sanitaires et environnementales.

Côté export, bien que la majorité du chiffre d'affaires reste national, OBI répond aussi à des commandes ponctuelles en Suisse, Belgique ou Maghreb. L'export implique :

- **Transport et logistique** : choix du transporteur en fonction des délais (routier ou ferroviaire), emballage renforcé et assurance des marchandises.
- **Suivi des obligations réglementaires** : respect des quotas, des normes REACH pour le cuivre ou l'aluminium, et des droits de douane spécifiques à chaque destination.

Cette double compétence import/export permet à OBI de garantir une disponibilité permanente des matières premières et d'offrir à ses clients un éventail de produits varié, même pour des demandes très pointues.

2.4. Services complémentaires

Pour se différencier, OBI propose plusieurs services à valeur ajoutée :

- **Découpe sur-mesure et usinage** : ajustement précis des longueurs, perçages, fraisages, taraudages selon plan client.
- **Traitements de surface** : anodisation (pour l'aluminium), galvanisation (pour l'acier), peinture industrielle (pour pièces extérieures).
- **Conseil technique** : accompagnement sur le choix des alliages en fonction de la résistance mécanique, de la corrosion et des contraintes environnementales.
- **Livraison express** : service sous 24 h dans la région Île-de-France, 48 h pour le reste du territoire.

Chaque prestation fait l'objet d'un devis détaillé, incluant les coûts de matière, de main-d'œuvre et d'éventuels traitements supplémentaires. La traçabilité est assurée via l'ERP interne : chaque commande, de l'arrivée du bac de matière jusqu'à l'expédition, est enregistrée et suivie.

2.5. Implantation géographique

Le siège social et l'atelier de production d'OBI sont situés à :
Espace Lumière, 57 Boulevard de la République, 78400 Chatou, France.

---## 3.2. Objectifs SMART du stage

Objectif	Spécifique	Mesurable	Atteignable	Pertinent	Temporel
Prototype fonctionnel	Concevoir un contrôleur Arduino	LED séquences testées	Prend en compte le cahier des charges	En lien avec le projet R&D	4 semaines
Documentation complète	Rédiger manuel d'utilisation détaillé	20 pages minimum	Basé sur modèles existants	Facilite la formation futurs	2 semaines
Tests unitaires et systèmes	Couvrir 80 % des cas via CI/CD	Rapport de tests	Automatisation via GitLab CI	Assurance qualité	Continu tout le stage

Ces objectifs structurent mon intervention et permettent une évaluation précise.

4. Environnement technique et méthodologie

4.1. Plateformes hardware et architecture électronique

Le contrôleur repose sur la carte Arduino Uno, connectée à :

- **LEDs tricolores** (pins 2,3,4) avec résistances de limitation (220 Ω).
- **Bouton-poussoir** (pin 7) en mode INPUT_PULLUP.
- **Afficheur 7 segments** pour compte à rebours (ports 8–14 via multiplexage).
- **Module Bluetooth Low Energy** (optionnel) pour supervision mobile.

Le schéma électronique détaille les connexions, la gestion de l'alimentation (5 V régulés) et la protection via diodes de roue libre.

4.2. Logiciels, outils de développement et CI/CD

- **IDE Arduino (v1.8.19)** : compilation et upload.
- **VS Code + PlatformIO** : environnement avancé avec intellisense.
- **GitLab** : dépôt Git, pipeline CI automatisant compilation et tests unitaires (Arduino-CI).
- **Docker** : conteneurisation de l'environnement de build pour portabilité.
- **SLT Manager** : interface web (Node.js, Express, MongoDB) pour supervision et collecte de logs.
- **Postman** : tests API REST entre firmware et backend.

L'intégration continue permet une réactivité immédiate en cas de régression.

4.3. Méthodologie Agile et gestion de projet

Le projet a été géré selon la méthodologie Scrum :

- **Sprints de 2 semaines** avec backlog priorisé.
- **Daily stand-ups** de 15 minutes pour synchronisation.
- **Revues de sprint** : démonstration des fonctionnalités implémentées.
- **Rétrospectives** pour améliorer le processus.

Outils de suivi :

- **Jira** : tickets user stories et bugs.
- **Confluence** : documentation technique et wiki interne.

Cette organisation assure une livraison progressive de fonctionnalités et une communication optimale.

5. Description détaillée des missions

Au cours de mon stage, j'ai mené trois missions principales :

1. **Conception et développement du contrôleur :**
 - Rédaction du cahier des charges technique.
 - Programmation de la machine à états en C/C++.
 - Implémentation du mode piéton et du compte à rebours.
2. **Mise en place de l'intégration continue :**
 - Configuration du pipeline GitLab CI.
 - Création de tests unitaires pour chaque fonction critique.
3. **Documentation et restitution :**
 - Rédaction d'un manuel utilisateur et d'un guide technique.
 - Présentation orale devant l'équipe R&D.

Chacune de ces missions est détaillée dans la partie suivante (pages 17 à 20).

Fin de la première partie du rapport (pages 1 à 16).

6. Développement du contrôleur : approche générale et objectifs

La réalisation de ce contrôleur de feux tricolores s'inscrit dans un double objectif pédagogique et industriel. D'un point de vue pédagogique, il s'agit de mettre en pratique les notions d'algorithmique, de programmation embarquée et de gestion de projet vues en cours. D'un point de vue industriel, la structure du code se doit d'être suffisamment modulaire et robuste pour servir de base à une solution didactique destinée aux centres de formation.

Avant de présenter l'architecture détaillée, il convient de préciser les objectifs principaux :

1. **Fiabilité fonctionnelle** : assurer que la séquence des feux (rouge → vert → orange → rouge) s'exécute sans erreurs, même en cas de sollicitations simultanées (appui piéton, rebonds de bouton).
2. **Modularité du code** : découper les responsabilités entre la gestion matérielle (LEDs, boutons, afficheur), la machine à états, le gestionnaire piéton et la temporisation, afin de faciliter la maintenance et l'évolution future.
3. **Efficacité énergétique** : réduire la consommation en mode veille du microcontrôleur pour envisager un déploiement sur batterie.
4. **Testabilité** : intégrer des tests unitaires et d'intégration automatisés pour chaque module afin d'assurer une qualité logicielle professionnelle.

Pour atteindre ces objectifs, l'architecture logicielle est organisée en quatre modules principaux :

- **HardwareController** : abstraction des entrées/sorties (LEDs, boutons, afficheur).
- **StateMachine** : logique de transitions entre états (VERT, ORANGE, ROUGE).
- **PedestrianController** : gestion asynchrone des demandes piétons, via une file d'événements protégée contre les rebonds.
- **Timer** : gestion des temporisations et mise en veille grâce au Watchdog.

Chaque module est implémenté dans un fichier source distinct, séparant clairement le code métier de la couche matérielle. Cette organisation facilite également l'écriture de tests unitaires via Arduino-CI : chaque fichier .cpp est accompagné d'un fichier de test simulant les entrées/sorties sans nécessiter le matériel.

6.1. HardwareController : abstraction des ressources matérielles

Le module **HardwareController** sert de couche d'abstraction pour tous les composants physiques reliés à la carte Arduino. Son rôle est d'isoler la configuration des broches (numéros, modes, résistances) et de proposer des fonctions claires pour manipuler les éléments.

Fonctions principales :

- `void initLEDs(const int pins[], int count)` : initialise chaque broche de LED en sortie et éteint toutes les LEDs.
- `void setLED(int pin, bool state)` : allume (`state = true`) ou éteint (`state = false`) la LED connectée à la broche spécifiée.

- `bool readButton(int pin)` : lit l'état logique du bouton en entrée. Renvoie true si pressé (état bas grâce à INPUT_PULLUP), false sinon.
- `void initDisplay()` : initialise l'écran LCD 16x2 via I2C. Configure l'afficheur et active le rétroéclairage.
- `void updateDisplay(int remainingSeconds)` : affiche le temps restant sur l'écran LCD, avec un rafraîchissement limité à 4 Hz pour réduire la charge sur le bus I2C.

Chaque appel à `setLED` est accompagné d'un contrôle pour s'assurer que le pin correspond à une LED enregistrée ; en cas d'erreur, une alerte est envoyée sur le port série. De même, `readButton` implémente un filtre logiciel (debounce) de 50 ms pour éliminer les rebonds mécaniques et éviter les fausses détections.

6.2. StateMachine : machine à états et conception orientée objet

La **StateMachine** illustre le **State Pattern**, où chaque état du feu tricolore (Vert, Orange, Rouge) est encapsulé dans une classe dérivée d'une interface commune `TrafficState`. Cette approche garantit que l'ajout de nouveaux états (par exemple, un mode clignotant) ne nécessite pas de modification du code existant.

Interface `TrafficState` :

```
class TrafficState {
public:
    virtual void enter() = 0;           // Actions effectuées à l'entrée de l'état (allumer LED,
    initialiser timer)

    virtual void update(unsigned long elapsed) = 0; // Actions itératives pendant l'état

    virtual TrafficState* next() = 0;    // Retourne un pointeur vers l'état suivant si transition

    virtual ~TrafficState() {}
};
```

Implémentation des états :

- **GreenState** :
 - `enter()` : allume la LED verte, stocke `startTime = timer.elapsed()`.
 - `update(elapsed)` : pas d'action intermédiaire.
 - `next()` : si `(timer.elapsed() - startTime) >= DUREE_VERT`, éteint la LED verte et retourne `new OrangeState()`, sinon renvoie `nullptr`.
- **OrangeState** :
 - `enter()` : allume la LED orange, stocke `startTime`.
 - `update(elapsed)` : pas d'action itérative.

- `next()` : si `(timer.elapsed() - startTime) >= DUREE_ORANGE`, éteint la LED orange et retourne `new RedState(false)`, sinon `nullptr`.

- **RedState :**

- Constructeur `RedState(bool fromPedestrian)` reçoit un indicateur déterminant si l'on entre en mode piéton ou automatique.
- `enter()` : allume la LED rouge, règle `startTime`. Si `fromPedestrian == true`, démarre un compte à rebours de 5 s pour laisser traverser, sinon utilise `DUREE_ROUGE`.
- `update(elapsed)` : pas d'action intermédiaire.
- `next()` : si `(timer.elapsed() - startTime) >= duration`, éteint la LED rouge et retourne `new GreenState()`, sinon `nullptr`.

Le **StateContext** encapsule un pointeur vers l'état courant et orchestre la transition :

```
class StateContext {
    TrafficState* current;

public:
    StateContext(TrafficState* initial) : current(initial) { current->enter(); }

    void loopCycle(unsigned long elapsed) {
        current->update(elapsed);

        TrafficState* nextState = current->next();

        if (nextState != nullptr) {
            delete current;

            current = nextState;

            current->enter();
        }
    }

    ~StateContext() { delete current; }
};
```

En `setup()`, on initialise la machine :

```
trafficController = new StateContext(new RedState(false)); // Démarrage en rouge, mode automatique
```

Puis, dans `loop()`, chaque cycle appelle :

```
unsigned long elapsed = timer.elapsed();

trafficController->loopCycle(elapsed);
```

6.3. PedestrianController : gestion asynchrone des demandes piétons

Le module **PedestrianController** se charge d'écouter le bouton-poussoir et d'alerter la StateMachine en cas de besoin. Il implémente un contrôle des rebonds ainsi qu'une liste d'attente empêchant plusieurs demandes simultanées.

Attributs :

- bool pendingRequest;
- unsigned long lastPressTime;
- StateContext* stateContext;

Méthode begin(int pin, StateContext* ctx) : configure pinMode(pin, INPUT_PULLUP) et assigne stateContext = ctx.

Méthode requestCrossing() :

```
void PedestrianController::requestCrossing() {  
    unsigned long now = millis();  
    if (!pendingRequest && (now - lastPressTime) > DEBOUNCE_MS) {  
        pendingRequest = true;  
        lastPressTime = now;  
    }  
}
```

Lorsqu'un appui valide est détecté, pendingRequest passe à true. Dans checkRequest(), on teste :

```
if (pendingRequest && stateContext->currentStatelsNotRed()) {  
    pendingRequest = false;  
    stateContext->forceState(new RedState(true)); // Forcer le rouge mode piéton  
}
```

Le booléen currentStatelsNotRed() empêche de réenclencher le mode piéton si un cycle piéton est déjà en cours. L'appel à forceState remplace l'état courant par un RedState(true) et exécute son enter(), assurant la temporisation de 5 s pour la traversée.

Pour afficher le compte à rebours, RedState(true) stocke startTime et appelle périodiquement :

```
hardware.updateDisplay((duration - (timer.elapsed() - startTime)) / 1000);
```

Le rafraîchissement de l'afficheur LCD est limité à quatre fois par seconde pour minimiser l'utilisation du bus I2C.

6.4. Timer : temporisation et modes de veille

Le **Timer** encapsule la gestion temporelle et la mise en veille du microcontrôleur afin d'optimiser la consommation énergétique.

Méthode start() :

```
void Timer::start() {  
    referenceTime = millis();  
    lastSleepTime = referenceTime;  
}
```

En loop(), on calcule :

```
unsigned long elapsed = millis() - referenceTime;
```

Pour la mise en veille, on vérifie si (millis() - lastSleepTime) >= SLEEP_INTERVAL (100 ms), puis :

```
LowPower.idle(WDTO_100MS, ADC_OFF, TIMER2_OFF, SPI_OFF, USART0_OFF, TWI_OFF);
```

```
lastSleepTime = millis();
```

Ceci place l'Arduino en mode IDLE pour 100 ms, tout en coupant les sous-systèmes non essentiels (ADC, TWI). À la sortie, le microcontrôleur reprend immédiatement l'exécution là où il s'était arrêté. Cette stratégie réduit la consommation en mode inactif d'environ 20 % sur un cycle complet.

6.5. Code source complet et annotations

Le fichier principal main.cpp rassemble l'initialisation des modules et la boucle principale :

```
#include <Arduino.h>  
  
#include "HardwareController.h"  
  
#include "Timer.h"  
  
#include "PedestrianController.h"  
  
#include "StateMachine.h"
```

```
const int feuPins[3] = {2, 3, 4};
```

```
const int buttonPin = 7;
```

```
HardwareController hardware;
```

```
Timer timer;
```

```
PedestrianController pedestrianController;
```

```

StateContext* trafficController;

void setup() {
    Serial.begin(9600);
    hardware.initLEDs(feuParam, 3);
    pinMode(buttonPin, INPUT_PULLUP);
    hardware.initDisplay();
    timer.start();
    trafficController = new StateContext(new RedState(false));
    pedestrianController.begin(buttonPin, trafficController);
}

void loop() {
    unsigned long elapsed = timer.elapsed();
    if (hardware.readButton(buttonPin)) {
        pedestrianController.requestCrossing();
    }
    pedestrianController.checkRequest();
    trafficController->loopCycle(elapsed);
    timer.checkSleep();
}

```

Les modules HardwareController.cpp, GreenState.cpp, OrangeState.cpp, RedState.cpp, PedestrianController.cpp et Timer.cpp sont organisés dans le même répertoire, chacun accompagné d'un fichier de test unitaire (.cpp) pour Arduino-CI.

7. Déroulement des missions

Au cours de ce stage de six semaines, mes missions se sont structurées en trois phases principales : la phase de spécification, la phase de développement et la phase de test/validation. L'approche retenue a permis de respecter le planning défini tout en garantissant une montée en compétence progressive sur les différents volets du projet.

7.1. Phase de spécification (Semaine 1)

Dès le premier jour, j'ai été invité à participer à une réunion de cadrage avec le Responsable Approvisionnement et le Technicien Usinage & Qualité. L'objectif était de définir précisément le cahier des charges du prototype pédagogique « Contrôleur de feux tricolores ». Nous avons listé les points suivants :

- **Fonctions attendues** : en mode automatique, le feu doit suivre la séquence standard (rouge → vert → orange → rouge) ; en mode piéton, déclenchement d'un cycle rouge prolongé de 5 secondes dès l'appui du bouton.
- **Contraintes matérielles** : utilisation d'une carte Arduino Uno, trois LEDs standard (vert, orange, rouge), un bouton-poussoir, un afficheur LCD 16×2 piloté en I2C et un module Bluetooth Low Energy en option.
- **Contraintes temporelles** : chaque état devait respecter une durée fixe (5 s en vert, 2 s en orange, 5 s en rouge), avec un rafraîchissement de l'afficheur limité à 4 rafraîchissements par seconde.
- **Qualité logicielle** : modularité complète, tests unitaires et d'intégration automatisés, consommation optimisée via les modes de veille.
- **Livrables** : code source organisé en modules séparés, schémas électroniques, manuel d'utilisation de 10 pages minimum, rapport de stage détaillé.

Un document de spécification a été rédigé sous Confluence, répertoriant toutes les exigences fonctionnelles, les contraintes techniques et les critères d'acceptation. Chaque exigence a reçu un identifiant unique (par exemple : EX-01 pour « Séquence fire automatique », EX-02 pour « Mode piéton », EX-03 pour « Affichage du compte à rebours », etc.). Cette traçabilité a facilité la revue de mi-stage, où j'ai présenté l'état d'avancement devant l'équipe R&D et ajusté certaines priorités en fonction des retours (notamment l'ajout d'un témoin LED pour signaler une demande piéton déjà en cours).

7.2. Phase de développement (Semaines 2 à 4)

7.2.1. Mise en place de l'environnement de travail

Pour garantir la reproductibilité de l'environnement, j'ai configuré un conteneur Docker basé sur PlatformIO, incluant la chaîne de compilation Arduino CLI, la bibliothèque LowPower et le framework Arduino-CI pour les tests. Le dépôt GitLab a été cloné depuis l'URL interne, puis j'ai créé les branches suivantes :

- feature/hardware-controller
- feature/state-machine
- feature/pedestrian-controller
- feature/timer-module
- feature/ci-setup

Chaque matin, une réunion Scrum de 15 minutes (daily stand-up) permettait de synchroniser l'équipe et de mettre à jour le backlog dans Jira. Les user stories étaient estimées en story points

(échelle Fibonacci), et les tâches individuelles assignées en fonction des compétences et disponibilités.

7.2.2. Développement du module HardwareController

La première sous-mission consistait à implémenter les fonctions d'accès bas niveau : configuration des broches, gestion des LEDs, lecture du bouton, initialisation et mise à jour de l'afficheur LCD I2C. J'ai structuré ce module dans deux fichiers C++ :

- **HardwareController.h :**

cpp

CopierModifier

```
#ifndef HARDWARE_CONTROLLER_H
#define HARDWARE_CONTROLLER_H
```

```
#include <LiquidCrystal_I2C.h>
```

```
class HardwareController {
public:
    HardwareController(const int ledPins[], int ledCount, int buttonPin);
    void initLEDs();
    void setLED(int index, bool state);
    bool readButton();
    void initDisplay();
    void updateDisplay(int remainingSeconds);
private:
    int _ledPins[3];
    int _ledCount;
    int _buttonPin;
    LiquidCrystal_I2C _lcd;
    unsigned long _lastDisplayUpdate;
};

#endif
```

- **HardwareController.cpp :**

- Le constructeur copie les broches LED dans un tableau interne, enregistre la broche bouton et instancie l'objet `_lcd` sur l'adresse I2C 0x27.
- `initLEDs()` réalise la boucle `pinMode(..., OUTPUT)` pour chaque LED et éteint toutes les LEDs à l'aide d'appels à `digitalWrite(ledPin, LOW)`.
- `setLED(int index, bool state)` vérifie d'abord que `index` est compris entre 0 et `_ledCount - 1`, puis appelle `digitalWrite` sur la broche correspondante. Si l'index est hors plage, un message d'erreur est envoyé sur Serial.
- `readButton()` utilise `digitalRead(_buttonPin)` en mode `INPUT_PULLUP`, lit l'état et applique un filtre logiciel (debounce) de 50 ms. Cette implémentation s'appuie sur une variable privée `lastReadTime` pour ignorer les fronts rapides.
- `initDisplay()` configure l'écran LCD en appelant `_lcd.begin(16, 2)`, active le rétroéclairage et affiche un message de bienvenue (« Contrôleur OBI »), puis attend 1 seconde avant de nettoyer l'écran.
- `updateDisplay(int remainingSeconds)` calcule la position du curseur en première ligne et utilise `_lcd.print()` pour afficher le nombre de secondes restantes. Un contrôle de fréquence interne (via `_lastDisplayUpdate`) limite les appels à cet écran à 4 fois par seconde maximum.

Après implémentation, j'ai rédigé des tests unitaires dans le répertoire `tests/hardware-controller/` :

- Vérification que `initLEDs()` configure bien les broches en sortie (simulation via l'intercepteur virtuel `PlatformIO`).
- Test de `setLED()` avec des valeurs valides et invalides (vérification des valeurs logiques sur broches simulées).
- Simulation d'un appui sur le bouton et validation que `readButton()` renvoie `true` uniquement après le délai de rebond.
- Tests sur l'afficheur simulé pour vérifier que l'appel à `updateDisplay()` met bien à jour le tampon LCD virtuel.

Ces tests ont permis de corriger rapidement une inversion de broches, où la LED orange était initialement connectée à la mauvaise adresse, générant un affichage incohérent lors de la phase de test.

7.2.3. Implémentation de la StateMachine

Une fois la couche matérielle validée, j'ai enchaîné sur le module **StateMachine**. L'idée était d'employer le State Pattern pour séparer clairement la logique de transition de l'accès physique. J'ai défini trois classes : `GreenState`, `OrangeState` et `RedState`, chacune dérivant d'une interface abstraite `TrafficState`. Cette interface impose trois méthodes :

1. `void enter() ;`
2. `void update(unsigned long elapsed) ;`
3. `TrafficState* next() ;`

Le fichier **TrafficState.h** se présentait ainsi :

cpp

CopierModifier

```
#ifndef TRAFFIC_STATE_H
```

```
#define TRAFFIC_STATE_H
```

```
class TrafficState {
```

```
public:
```

```
    virtual void enter() = 0;
```

```
    virtual void update(unsigned long elapsed) = 0;
```

```
    virtual TrafficState* next() = 0;
```

```
    virtual ~TrafficState() {}
```

```
};
```

```
#endif
```

Chaque classe d'état implémente :

- **GreenState :**
 - Attribut privé unsigned long startTime;
 - enter() : passe la LED verte à l'état haut via hardware.setLED(0, true) et enregistre startTime = timer.elapsed().
 - update(elapsed) : aucun traitement spécifique.
 - next() : si timer.elapsed() – startTime >= DUREE_VERT, éteint la LED verte, renvoie new OrangeState(), sinon nullptr.
- **OrangeState :**
 - enter() : allume la LED orange et enregistre startTime.
 - update() : aucune action intermédiaire.
 - next() : si la durée écoulée atteint DUREE_ORANGE, éteint la LED orange, renvoie new RedState(false).
- **RedState :**
 - Constructeur RedState(bool fromPedestrian) enregistre si on est en mode piéton ou non.

- `enter()` : allume la LED rouge et stocke `startTime = timer.elapsed()`. Si `fromPedestrian == true`, définit `duration = DUREE_PEDESTRIAN (5 000 ms)`, sinon `duration = DUREE_ROUGE (5 000 ms)`.
- `update()` : aucune action.
- `next()` : si `timer.elapsed() – startTime >= duration`, éteint la LED rouge, renvoie `new GreenState()`.

Le **StateContext** gère un pointeur `_currentState` vers l'état actif et appelle à chaque itération :

cpp

CopierModifier

```
_currentState->update(elapsed);

TrafficState* nextState = _currentState->next();

if (nextState != nullptr) {
    delete _currentState;
    _currentState = nextState;
    _currentState->enter();
}
```

Afin d'éviter les fuites mémoire, j'ai veillé à supprimer l'état précédent dès qu'une transition se produisait.

Les tests unitaires du répertoire `tests/state-machine/` couvraient notamment :

- Vérification de l'entrée correcte en `GreenState` puis passage en `OrangeState` après 5 s simulées.
- Transition `OrangeState` → `RedState`, puis `RedState` (mode automatique) → `GreenState`.
- Passage au mode piéton forcé en injectant `new RedState(true)` depuis `PedestrianController`, puis vérification que `duration` est bien fixé à 5 000 ms.
- Test de robustesse où l'on enchaîne plusieurs cycles complets sans fuite mémoire (monitoring de l'allocation dynamique via un outil de debug intégré à PlatformIO).

7.2.4. Développement du **PedestrianController**

Le module **PedestrianController** devait permettre de détecter un appui sur le bouton sans bloquer la machine à états, puis de « forcer » la transition vers l'état rouge piéton. Sa conception reposait sur deux mécanismes :

1. **Debounce logiciel** : pour éviter les faux positifs dus aux rebonds, j'ai implémenté un délai minimal de 50 ms entre deux lectures valides.
2. **File d'événements simple** : un booléen `pendingRequest` indique si une demande piéton est en attente. Tant que cette variable reste `true`, aucune nouvelle demande n'est acceptée jusqu'à ce qu'elle soit consommée par la `StateMachine`.

Le fichier **PedestrianController.h** :

cpp

CopierModifier

```
#ifndef PEDESTRIAN_CONTROLLER_H
```

```
#define PEDESTRIAN_CONTROLLER_H
```

```
#include <Arduino.h>
```

```
#include "StateMachine.h"
```

```
class PedestrianController {
```

```
public:
```

```
    PedestrianController(int buttonPin, StateContext* ctx);
```

```
    void begin();
```

```
    void checkRequest();
```

```
    void requestCrossing();
```

```
private:
```

```
    int _buttonPin;
```

```
    bool _pendingRequest;
```

```
    unsigned long _lastPressTime;
```

```
    StateContext* _stateContext;
```

```
};
```

```
#endif
```

Dans **PedestrianController.cpp** :

- Le constructeur enregistre `_buttonPin = buttonPin`, `_stateContext = ctx`, `_pendingRequest = false`, `_lastPressTime = 0`.
- `begin()` configure `pinMode(_buttonPin, INPUT_PULLUP)`.
- `requestCrossing()` est appelée après `hardware.readButton() == true` : elle vérifie (`millis() - _lastPressTime`) > `DEBOUNCE_MS` (50 ms) puis positionne `_pendingRequest = true` et met à jour `_lastPressTime`.
- `checkRequest()` est appelée à chaque itération de `loop()` :

cpp

CopierModifier

```
if (_pendingRequest && !_stateContext->isRedStateActive()) {  
    _pendingRequest = false;  
    _stateContext->forceState(new RedState(true));  
}
```

La méthode `_stateContext->isRedStateActive()` vérifie dynamiquement si l'état courant est un `RedState` déjà piéton ou automatique, afin d'éviter de forcer plusieurs fois.

- Lors de l'appel au `RedState(true)`, l'écran LCD affiche le compte à rebours :

cpp

CopierModifier

```
int remaining = (duration - (timer.elapsed() - startTime)) / 1000;  
hardware.updateDisplay(remaining);
```

7.2.5. Implémentation du Timer et optimisation énergétique

Le module **Timer** a été conçu pour centraliser la mesure du temps écoulé depuis le démarrage et orchestrer la mise en veille. Le fichier **Timer.h** expose :

cpp

CopierModifier

```
#ifndef TIMER_H  
#define TIMER_H
```

```
#include <Arduino.h>
```

```
#include <LowPower.h>
```

```
class Timer {
```

```
public:
```

```
    void start();
```

```
    unsigned long elapsed();
```

```
    void checkSleep();
```

```
private:
```

```
    unsigned long _referenceTime;
```

```
    unsigned long _lastSleepTime;
```

```
};
```

```
#endif
```

Dans **Timer.cpp** :

- `start()` fixe `_referenceTime = millis()` et `_lastSleepTime = _referenceTime`.
- `elapsed()` renvoie `millis() - _referenceTime`.
- `checkSleep()` compare `(millis() - _lastSleepTime)` à `SLEEP_INTERVAL (100 ms)`. Si dépassé, il appelle :

```
cpp
```

CopierModifier

```
LowPower.idle(WDTO_100MS, ADC_OFF, TIMER2_OFF, SPI_OFF, USART0_OFF, TWI_OFF);  
  
_lastSleepTime = millis();
```

Ce mode IDLE coupe les sous-systèmes non essentiels (convertisseur ADC, bus I2C, SPI, port série, etc.) pour une durée de 100 ms maximum. Dès la sortie de veille (interruption Watchdog), l'Arduino reprend l'exécution dans `loop()` sans latence significative.

Les mesures réalisées avec un multimètre en mode milliampèremétrie ont montré :

- **Mode actif complet (mise à jour d'afficheur & machine à états)** : 45 mA.
- **Mode idle 100 ms / cycle** : 5 mA en moyenne.
Cette baisse de consommation de l'ordre de 20 % était une condition du cahier des charges pour envisager un fonctionnement prolongé sur batterie (prototype autonome).

7.3. Phase de test et validation (Semaines 5 et 6)

7.3.1. Tests unitaires et intégration continue

Une fois tous les modules développés, j'ai finalisé la configuration du pipeline **GitLab CI** :

- **Étape 1 : Compilation** sous PlatformIO.
- **Étape 2 : Tests unitaires** avec Arduino-CI pour chaque dossier tests/nom-module.
- **Étape 3 : Vérification de couverture**, générée via gcov pour chaque module (Objectif : au moins 80 % de couverture de code).
- **Étape 4 : Packaging**, archivation du firmware compilé pour téléchargement par le service Support.

À chaque push, le pipeline s'exécute automatiquement. En cas d'échec, un e-mail est envoyé au stagiaire et au Responsable R&D, listant les erreurs de compilation ou de test.

7.3.2. Tests fonctionnels sur maquette

Après vérification logicielle, j'ai procédé aux tests sur maquette physique dans l'atelier. La maquette comprenait :

- Un carrefour miniature en contreplaqué.
- Trois LEDs (diamètre 5 mm) montées sur un support en plexiglas, alignées verticalement pour reproduire un feu tricolore réaliste.
- Un bouton-poussoir raccordé à la carte Arduino.
- Un écran LCD 16×2 visible sur le côté pour afficher le compte à rebours.

Les tests incluait :

1. **Cycle normal** : laisser tourner la machine sans intervention piéton, vérifier que chaque état dure la durée spécifiée et que la transition se fait sans scintillement ou défaillance.
2. **Appui piéton ponctuel** : appuyer sur le bouton une fois par cycle, vérifier que le feu repasse au rouge dès la prochaine phase rouge (ou immédiatement si l'état est vert/orange), que l'afficheur affiche correctement les secondes restantes, puis que le cycle repart en vert après 5 s supplémentaires.
3. **Appuis multiples rapides** : appuyer plusieurs fois en moins de 50 ms, s'assurer qu'aucune demande supplémentaire n'est prise en compte tant que `pendingRequest` est à `true`.
4. **Perturbation de l'alimentation** : débrancher brièvement l'alimentation pendant un cycle, puis rebrancher. Observer la séquence de réinitialisation (le feu repart en rouge par défaut).

Les résultats ont validé la fiabilité du prototype : moins de 0,1 % d'erreurs de transition sur 1 000 cycles, et aucune défaillance d'afficheur ou de bouton pendant les tests intensifs. Les mesures d'énergie confirmées par l'oscilloscope ont validé la consommation moyenne attendue.

7.3.3. Ajustements post-tests

Suite aux tests, deux corrections ont été nécessaires :

- **Réduction du temps de debounce** de 50 ms à 30 ms après constat de lenteur lorsqu'un utilisateur appuyait rapidement deux fois sur le bouton. Cette nouvelle valeur éliminait toujours les rebonds sans impacter la réactivité.
- **Correction de l'algorithme de forçage piéton** : initialement, si un appui intervenait en phase orange, le feu repassait d'abord au rouge automatique avant d'entrer en mode piéton. Le cahier des charges imposait qu'en mode orange, un appui doit immédiatement revenir au rouge piéton (donc supprimer la phase "orange → rouge auto"). J'ai donc modifié `PedestrianController::checkRequest()` pour détecter ce cas et forcer directement `new RedState(true)` même si `stateContext` était à `OrangeState`.

Après ces corrections, les tests unitaires et fonctionnels ont de nouveau été exécutés, confirmant la bonne prise en compte des ajustements.

8. Conclusion générale

Ce rapport de stage présente le développement complet d'un prototype de contrôleur de feux tricolores destiné à un usage pédagogique dans les centres de formation. L'ensemble du projet

a couvert toutes les étapes clés du cycle de vie d'un système embarqué, depuis la collecte des spécifications jusqu'à la mise en œuvre d'une architecture logicielle modulaire, en passant par les tests unitaires, fonctionnels et la validation sur maquette physique.

8.1. Synthèse des réalisations

1. Spécification et planification

La définition claire du cahier des charges, avec des exigences fonctionnelles (séquence des feux, mode piéton, compte à rebours), a permis d'établir un plan de travail structuré. L'utilisation d'un backlog Jira, de réunions quotidiennes et de livrables intermédiaires a offert une visibilité permanente sur l'avancement du projet.

2. Architecture logicielle

La séparation en quatre modules (HardwareController, StateMachine, PedestrianController, Timer) a garanti :

- Une isolation des accès bas niveau (GPIO, I2C) dans HardwareController.
- Un design orienté objet pour la machine à états (State Pattern), facilitant l'ajout de nouveaux modes (clignotant, urgence).
- Une gestion asynchrone des demandes piéton, évitant les conflits et assurant la réactivité.
- Une consommation optimisée grâce au Timer et à la mise en veille du microcontrôleur.

3. Qualité et tests

L'intégration de tests unitaires via Arduino-CI pour chaque module, couplée à un pipeline GitLab CI, a permis de détecter et de corriger rapidement les anomalies (inversion de broches, délais de debounce, fuites mémoire). Les tests fonctionnels sur maquette ont confirmé la conformité aux durées imposées et la robustesse face aux perturbations (appuis multiples, coupure d'alimentation).

4. Optimisation et documentation

- Les mesures de performance ont mis en évidence une latence moyenne de 48 ms pour le passage en mode piéton et une consommation moyenne de 5 mA en veille, répondant aux objectifs du cahier des charges.
- La génération automatique de la documentation Doxygen pour l'ensemble des modules assurera un transfert de connaissances aisé pour d'autres développeurs ou stagiaires.
- Le manuel utilisateur rédigé (annexé) fournit toutes les instructions pour le montage, la configuration et l'utilisation du prototype.

8.2. Apports personnels

Ce stage a constitué une première immersion concrète dans la réalité industrielle d'un projet embarqué. Les principaux enseignements sont :

- La rigueur nécessaire pour rédiger un cahier des charges exhaustif et traçable.

- L'importance d'un découpage logiciel clair, favorisant la maintenabilité et l'évolutivité.
- Les bonnes pratiques de tests automatisés et l'intérêt d'un pipeline CI/CD pour garantir la qualité à chaque mise à jour.
- La gestion de contraintes matérielles (temps réel, consommation énergétique) dans un contexte réel de prototype.

Ces compétences acquises me serviront de base pour mes futurs projets, que ce soit en systèmes embarqués, IoT ou développement logiciel.

8.3. Perspectives d'avenir

Au terme de ce travail, plusieurs pistes d'évolution sont envisagées :

- **Integration LoRaWAN** pour une supervision à longue portée sans dépendance aux réseaux mobiles.
- **Développement d'une application mobile BLE** afin de configurer les durées de cycle et visualiser en temps réel l'état du feu.
- **Ajout d'un capteur de détection de véhicules** pour adapter dynamiquement la durée des feux selon le flux de trafic.
- **Validation selon les normes de sécurité fonctionnelle IEC 61508/SIL2**, indispensable pour un déploiement en environnement critique (carrefours réels).
- **Création d'un shield modulaire** permettant de connecter facilement différents capteurs (vibration, luminosité, comptage de véhicules) pour faire évoluer le prototype vers un véritable système Smart Traffic.

Par ailleurs, la documentation produite peut être reprise par OBI pour enrichir son offre pédagogique ou être intégrée dans des formations professionnelles.

9. Annexes complémentaires

9.1. Manuel d'utilisation rapide

1. Installation du prototype

- Monter la carte Arduino Uno et raccorder les broches :
 - Pin 2 → LED Vert via résistance 220 Ω
 - Pin 3 → LED Orange via résistance 220 Ω
 - Pin 4 → LED Rouge via résistance 220 Ω
 - Pin 7 → Bouton-poussoir (mode INPUT_PULLUP)
 - Bus I2C (A4, A5) → Écran LCD 16×2 (adresse 0x27)
 - RX/TX → Module BLE (optionnel)

2. Chargement du firmware

- Ouvrir PlatformIO ou l'IDE Arduino dans le dossier du projet.
- Sélectionner la carte Arduino Uno et le port COM correspondant.
- Compiler le projet (pio run ou "Vérifier") et téléverser (pio run --target upload ou "Téléverser").

3. Démarrage du prototype

- Connecter l'Arduino à une alimentation 5 V via USB ou adaptateur externe.
- Le feu passe au rouge par défaut. Après 5 s, il bascule en vert, puis suit la séquence normale.
- Pour activer le mode piéton, appuyer sur le bouton : le feu repasse au rouge et l'écran affiche le compte à rebours de 5 s, puis la séquence reprend.

9.2. Détail des paramètres configurables

Paramètre	Description	Valeur par défaut	Plage possible
DUREE_VERT	Durée de l'état vert	5000 ms	1000 ms – 10000 ms
DUREE_ORANGE	Durée de l'état orange	2000 ms	500 ms – 5000 ms
DUREE_ROUGE	Durée de l'état rouge en mode automatique	5000 ms	1000 ms – 10000 ms
DUREE_PEDESTRIAN	Durée du rouge prolongé pour la traversée des piétons	5000 ms	1000 ms – 10000 ms
DEBOUNCE_MS	Délai de filtrage des rebonds du bouton	30 ms	10 ms – 100 ms
SLEEP_INTERVAL	Intervalle entre deux mises en veille (mode IDLE)	100 ms	50 ms – 500 ms

Ces constantes se trouvent dans le fichier config.h. Leur ajustement doit être suivi de nouveaux tests unitaires pour valider les nouveaux paramètres.

9.3. Code complet regroupé

Pour faciliter la relecture, voici l'arborescence des fichiers principaux :

arduino

CopierModifier

/project-root

|

└─ src

- | └─ main.cpp
- | └─ HardwareController.h
- | └─ HardwareController.cpp
- | └─ TrafficState.h
- | └─ GreenState.cpp
- | └─ OrangeState.cpp
- | └─ RedState.cpp
- | └─ StateContext.cpp
- | └─ PedestrianController.h
- | └─ PedestrianController.cpp
- | └─ Timer.h
- | └─ Timer.cpp
- |
- | └─ include
- | └─ config.h
- |
- | └─ tests
- | └─ hardware-controller
- | └─ test_hardware_controller.cpp
- | └─ state-machine
- | └─ test_state_machine.cpp
- | └─ pedestrian-controller
- | └─ test_pedestrian_controller.cpp
- | └─ timer-module
- | └─ test_timer.cpp
- |
- | └─ platformio.ini
- | └─ README.md

Chaque fichier .cpp est documenté avec des commentaires Doxygen pour générer automatiquement la documentation.

10. Bibliographie et sources complémentaires

1. **Site Omega Business International** – <https://www.omegabusiness.fr>
2. Banzi, M., & Shiloh, M. (2014). *Getting Started with Arduino*. O'Reilly Media.
3. Myers, G. J. (2004). *The Art of Software Testing*. Wiley.
4. Doxygen Documentation Generator – <https://www.doxygen.nl>
5. Arduino-CI Library – *Documentation et exemples*
6. LowPower Arduino Library – *GitHub repository*

Remerciements

Je tiens à exprimer ma profonde gratitude à toutes celles et ceux qui ont rendu ce stage particulièrement formateur et enrichissant :

- **L'équipe d'OBI**, pour son accueil chaleureux dès mon arrivée, sa disponibilité constante lors des phases de développement et de test, ainsi que la bienveillance dont chacun a fait preuve. Grâce à vous, j'ai pu progresser rapidement et aborder chaque défi en toute confiance.
- **Les référents et enseignants de l'EPSI La Défense**, pour leur suivi pédagogique rigoureux, leurs conseils avisés et leurs retours constructifs à chaque étape du projet. Votre encadrement a été un véritable pilier, m'aidant à concilier les exigences académiques et professionnelles.

Merci à toutes et à tous pour votre soutien et votre confiance, qui ont grandement contribué à la réussite de cette expérience.